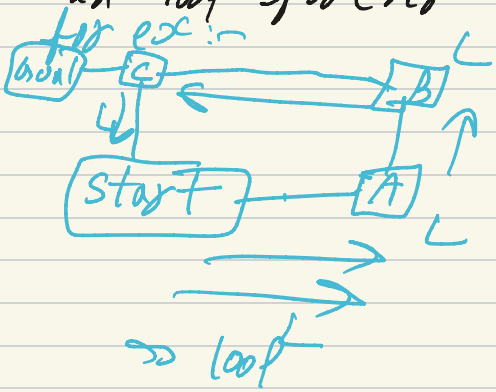
Made with Goodnotes

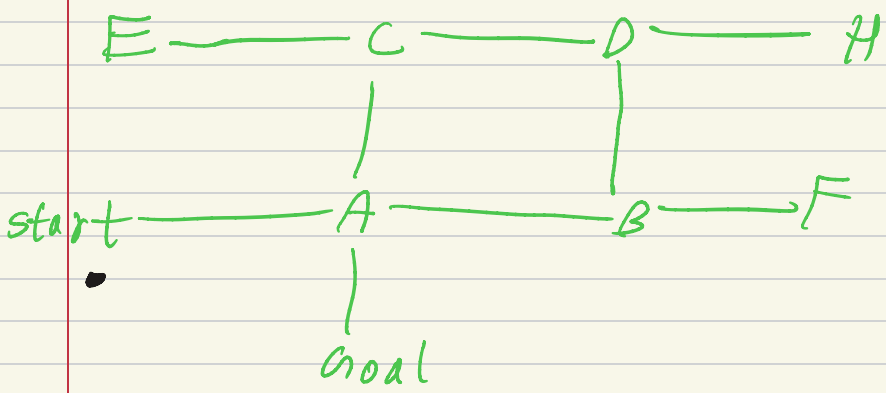
DFS	Wall-following
– At each junction take an un-explored branch, at dead-end return to the last point, record visited junctions and end-points	– Always prefer one side, no memory is needed.
– Advantages:- A more powerful algo, builds a map that can be re-used.	– Advantages:- Literally small code and no memory reqd.
– Disadvantages:- Needs memory, better detection systems needed.	– Disadvantages:- Can loop forever.

for ex:-



I would prefer to use DFS for an unknown maze, due to the fact that it has almost surety to find the correct path.

Made with Goodnotes



Step	Path	Vectors
1.)	S → A	(S)
2.)	A → C	(S, A)
3.)	C → E	(S, A, C)
4.)	E → C	(S, A)
5.)	C → D	(S, A, C)
6.)	D → H	(S, A, C, D)
7.)	H → D	(S, A, C)
8.)	D → B	(S, A, C, D)
9.)	B → F	(S, A, C, D, B)
10.)	F → B	(S, A, C, D)
11.)	B → A	(S)
12.)	A → Goal	(S, A)

In exploration the robot runs for a long time (ideally), but when it does the replay run, it does it blazing fast as it just needs to follow a path.

Made with Goodnotes